I-Logical Approaches Propositional Syntax and Tensor Network Decompositions

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

March 30, 2026

Outline

- Motivation: Propositional syntax
- Basis calculus
- Implementations in `tnreason`

Decomposition of formulas

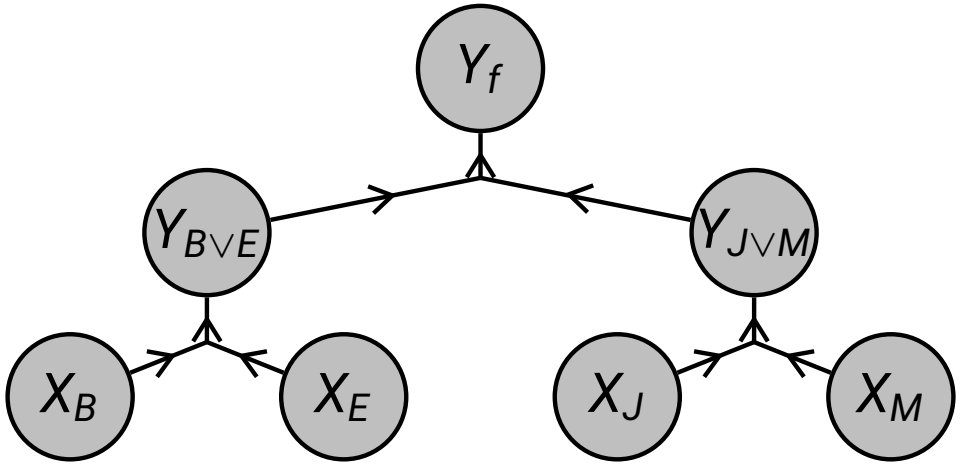
Recall: Tensor networks as a sparsity principle

- Knowledge base decomposition into formulas
- Can we exploit further sparsity in the syntax of the formulas?

Example: The formula *"When a burglary or an earthquake occurs, either our neighbor John or Mary call."* has a representation by

$$f\,[X_B, X_E, X_J, X_M] = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$$

Intuitively we can decompose the

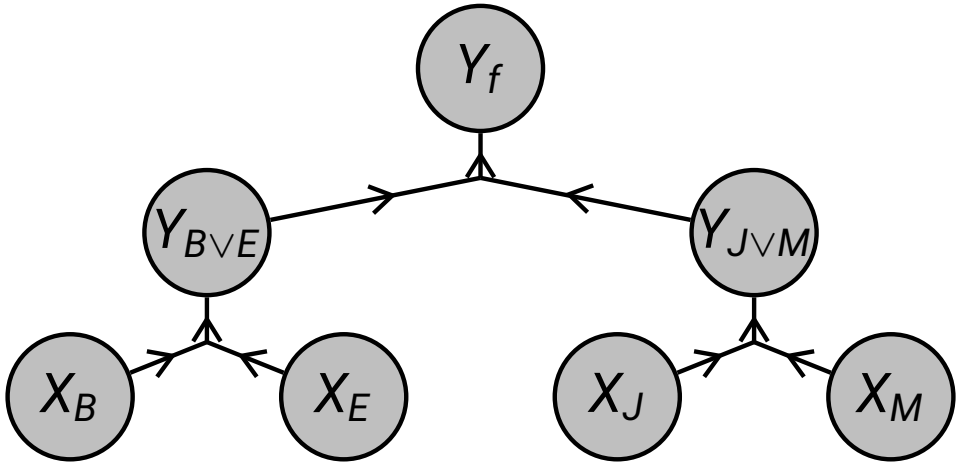


Decomposition of formulas

Example: The formula *"When a burglary or an earthquake occurs, either our neighbor John or Mary call."* has a representation by

$$f\,[X_B, X_E, X_J, X_M] = (X_B \vee X_E) \Rightarrow (X_J \vee X_M)$$

Intuitively we can decompose the



In this lecture we:

- understand each Y as a hidden variable
- decorate directed hyperedges by basis encodings to find tensor network decompositions

How to represent formulas as tensor networks?

Multilinearity of tensor networks

Tensor networks are multilinear, that is linear in each core tensor decorating an edge.

The logical conjunction is the only connective function which is multilinear.

Non-linearity problem

How to represent non-linear formulas like $\neg X$ in a tensor network?

Solution:
We develop the basis calculus scheme instead of coordinate calculus.

Outline

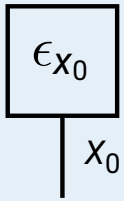
- Motivation: Propositional syntax
- Basis calculus
- Implementations in `tnreason`

Recall: One-hot encodings

One-hot encodings of a state are a tensor with a single coordinate 1 marking the corresponding state.

One-hot encodings

For a state $x \in [m]$ of a variable X the one-hot encoding is



$$= \begin{pmatrix} 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \end{pmatrix} [X_0]$$

$$\begin{array}{ccc} \vdots & & \vdots & & \vdots \\ 0 & & x_0 & & m_0 - 1 \end{array}$$

Given a factored representation with multiple variables $X_{[d]}$, the one-hot encoding of a state $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}} \left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k} \left[X_k\right] . \tag{1}$$

They build an orthogonal basis of the tensor space.

Coordinate and basis encoding

Coordinate encoding:
Store an information $f(x_{[d]})$ in the coordinates of a tensor

$$f\left[X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} f(x_{[d]}) \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Basis encoding:
Store an information $f(x_{[d]})$ in a basis vector $\epsilon_{f(x_{[d]})}\left[Y_f\right]$ to an auxiliary variable Y_f, that is

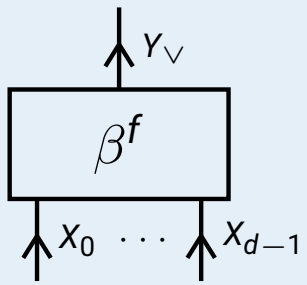
$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})}\left[X_{[d]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Basis encodings

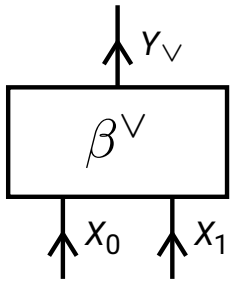
Definition (Basis encoding)

The basis encodings of a formula $f\left[X_{[d]}\right] : \{0,1\}^d \to \{0,1\}$ is the tensor

$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})}\left[Y_f\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \quad ,$$



Example: The basis encoding of the logical disjunction $\vee$ is the tensor



$$= \quad \begin{array}{c} X_1 \\ \begin{array}{c} 0 \\ 1 \end{array} \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{array}{c} 0 \\ 1 \end{array} \\ X_0 \end{array} \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{array}{c} 1 \\ 0 \\ Y_\vee \end{array} \qquad [X_0, X_1, Y_\vee]$$

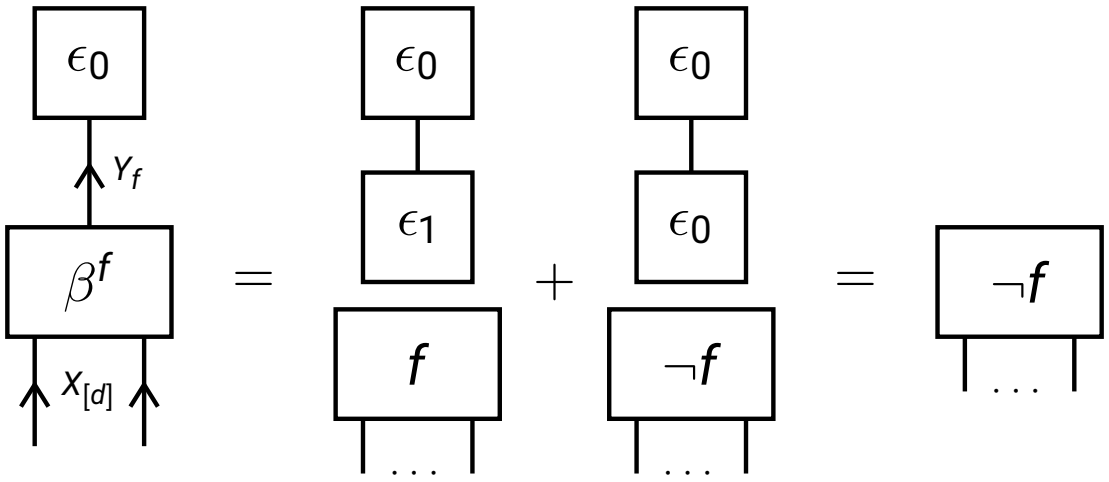
Solution of the non-linearity problem

Basis encodings resolve the nonlinearity problem: Any composition of formulas can be represented as a tensor network.

Example: Logical negation of a formula

$$\neg f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \epsilon_0\left[Y_f\right]\right\rangle_{\left[X_{[d]}\right]}$$

by



Basis calculus perspective on logics

Boolean states are represented by one-hot encodings

$$\mathrm{True} \leftrightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix} [X] \quad \text{and} \quad \mathrm{False} \leftrightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix} [X] \ .$$

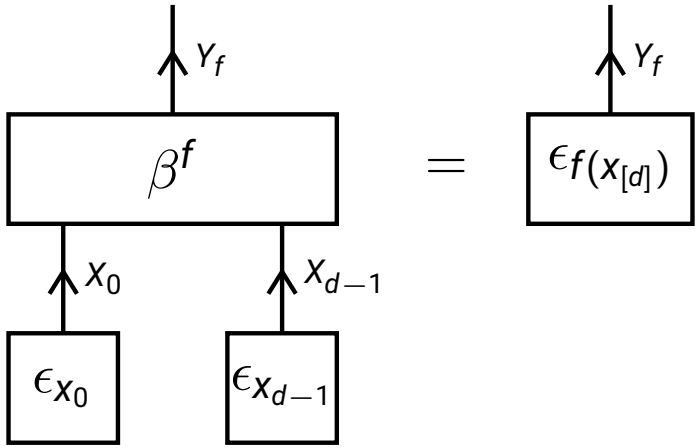
The truth tables of logical connectives $\circ \in \{\wedge, \vee, \Rightarrow\}$ are represented by tensors $\beta^\circ \in \mathbb{R}^{2\times 2\times 2}$ defined as
Contracting with the one-hot encoding of the inputs gives the one-hot encoding of the formula satisfaction.

Basis calculus

Idea of basis calculus:

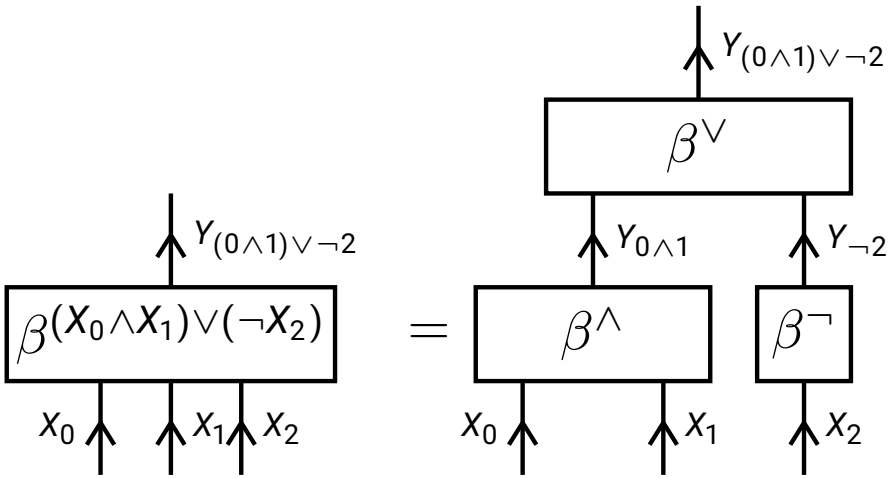
- Represent input $x_{[d]}$ by one-hot encoding $\epsilon_{x_{[d]}}\left[X_{[d]}\right]$
- Contract with the basis encoding of a function
- Get one-hot encoding of $f(x_{[d]})$ as a result

For any $x_{[d]}$ it holds that:



Decomposition of basis encodings

Example of a decomposition:

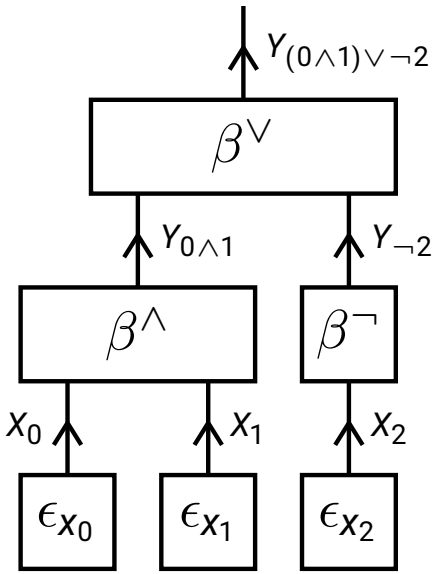


Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

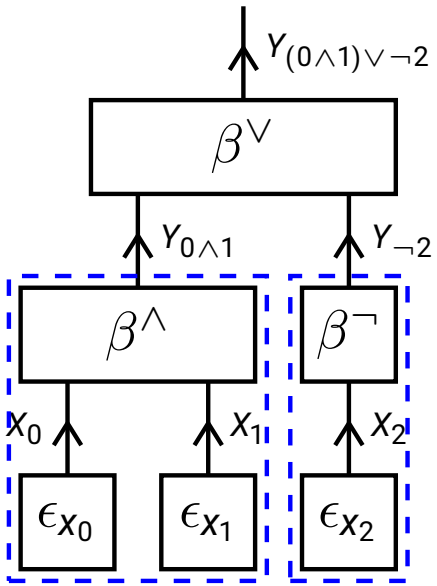


Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

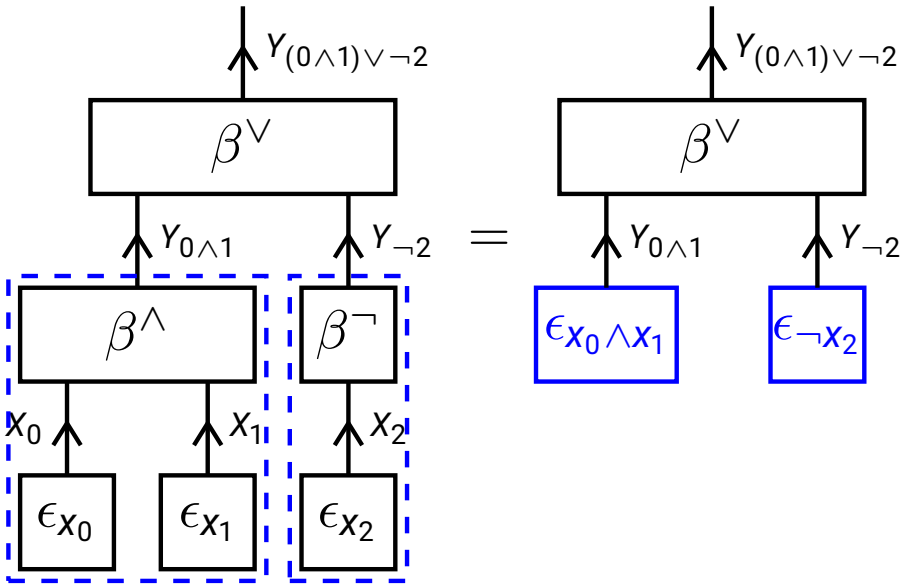


Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

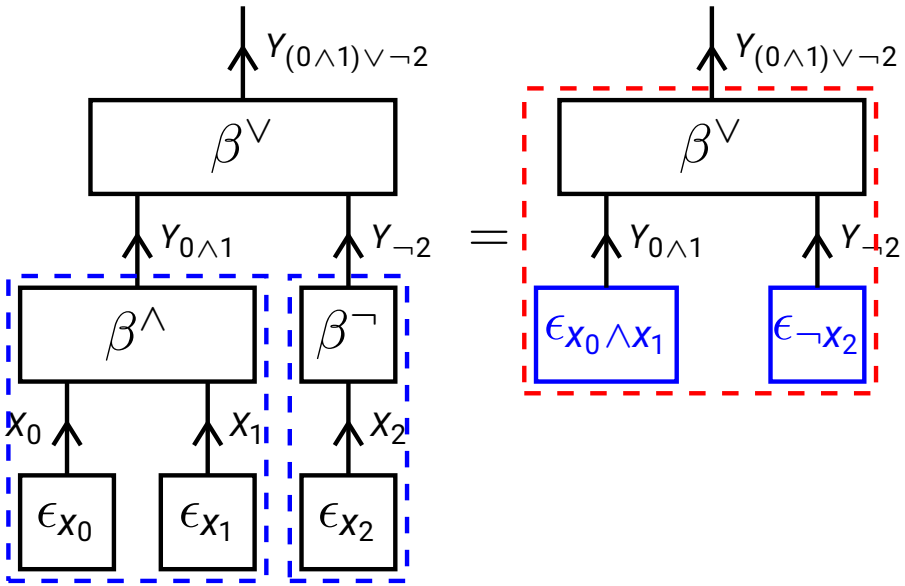


Formula evaluation by message passing of one-hot encodings

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

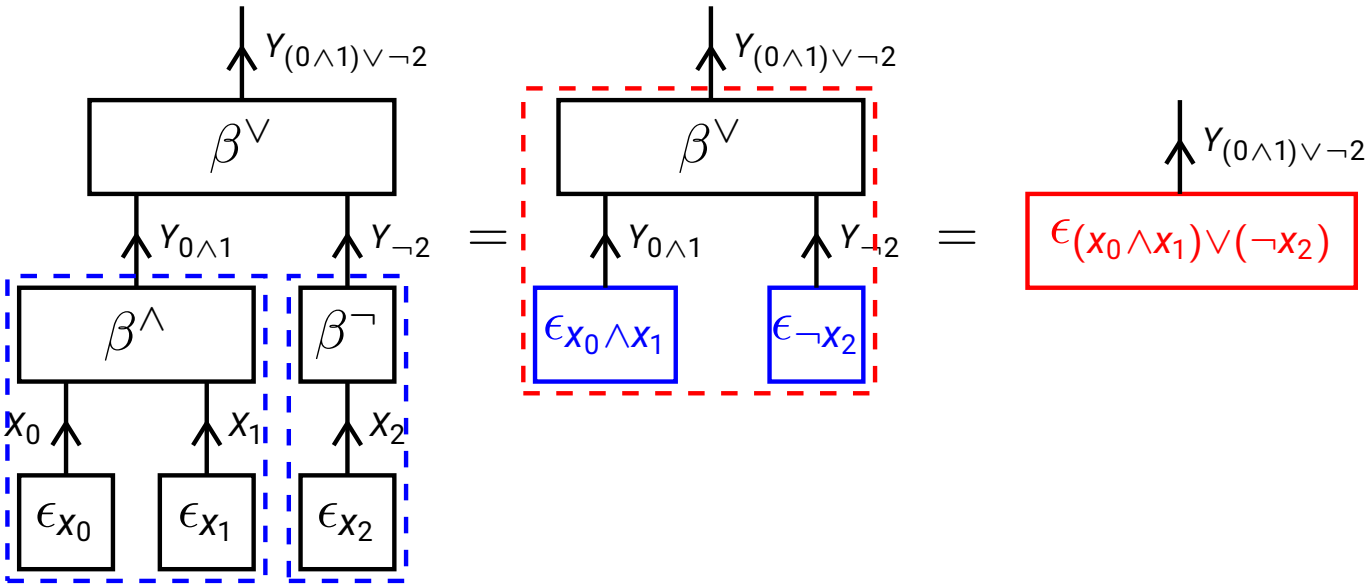


Formula evaluation by message passing of one-hot encodings

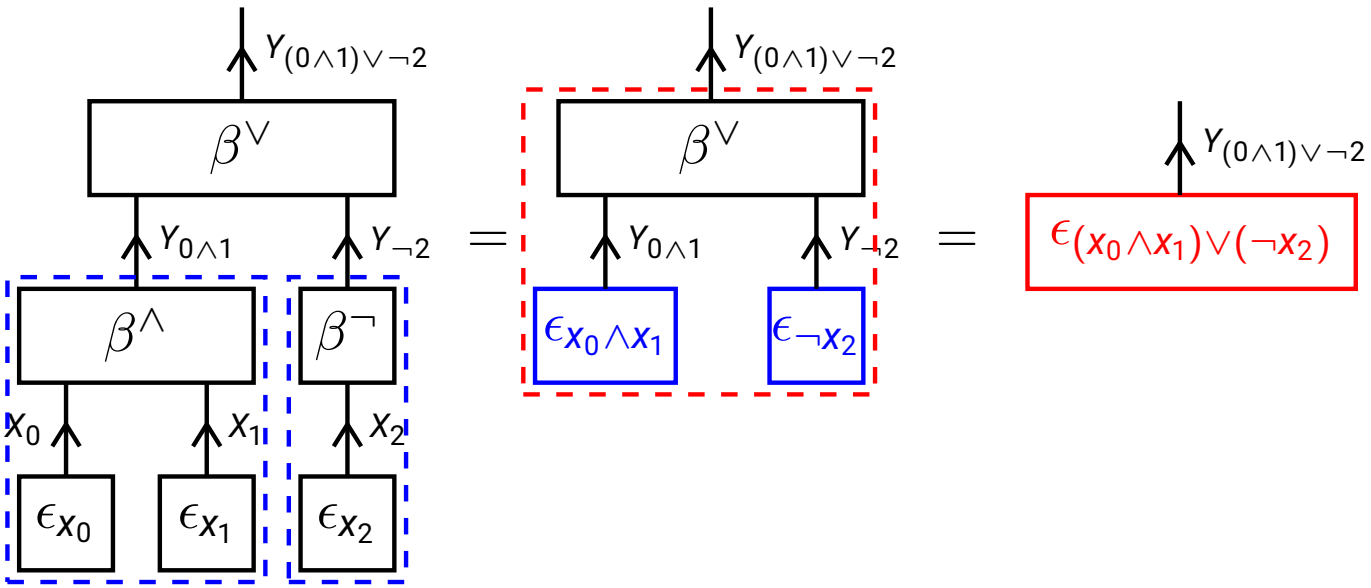
Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.



Formula evaluation by message passing of one-hot encodings



This is a first example of a message passing scheme:

- "Messages" are computed by contractions of sub-networks (two blue contractions, one red contraction)
- "Passing" of previous messages (blue contractions) to compute new messages (red contraction)

Decomposition sparsity

Theorem

The basis encoding of a composed formula is the contraction of the basis encodings to its connectives.

Proof: When contracting the decomposition with one-hot encodings of any $x_{[d]} \in [2]^d$, we get by basis calculus the one-hot encodings of the composition formulas, and finally the one-hot encoding of the whole formula. This coincides with contracting the full basis encoding since

$$\left\langle \beta^f \left[Y_f, X_{[d]}\right], \epsilon_{x_{[d]}} \left[X_{[d]}\right] \right\rangle_{[Y_f]} = \epsilon_{f\left[X_{[d]}=x_{[d]}\right]} \left[Y_f\right] .$$

Advantages of the Network Decomposition

Model-theoretic Semantics by Tensor Networks

- **Tensors encode the models** of propositional formulas by their nonzero coordinates
- **Networks eliminate redundancies** when representing similar formulas

This is an implementation of the **neural paradigm**, resulting in

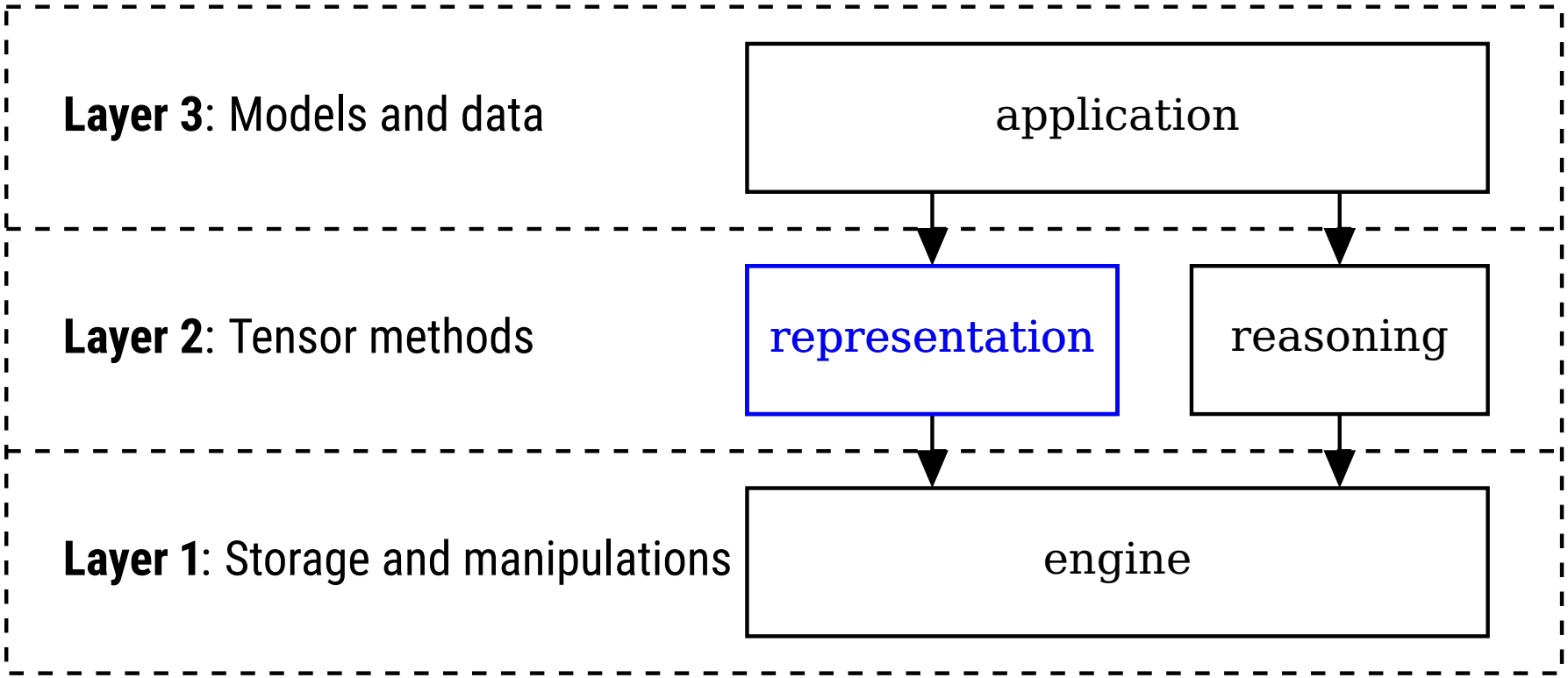
- Effective representation: Latent variables by logical formulas
- Differential parametrization: Multilinearity with respect to each core tensor

Outline

- Motivation: Propositional syntax
- Basis calculus
- Implementations in `tnreason`

Implementations in tnreason**:** **Subpackage** representation

The subpackage representation is dedicated to the tensor encoding schemes such as basis encodings.



Subpackage representation**: Basis encodings**

Creation of a basis encoding $\beta^f\left[Y_f, X_{[d]}\right]$:

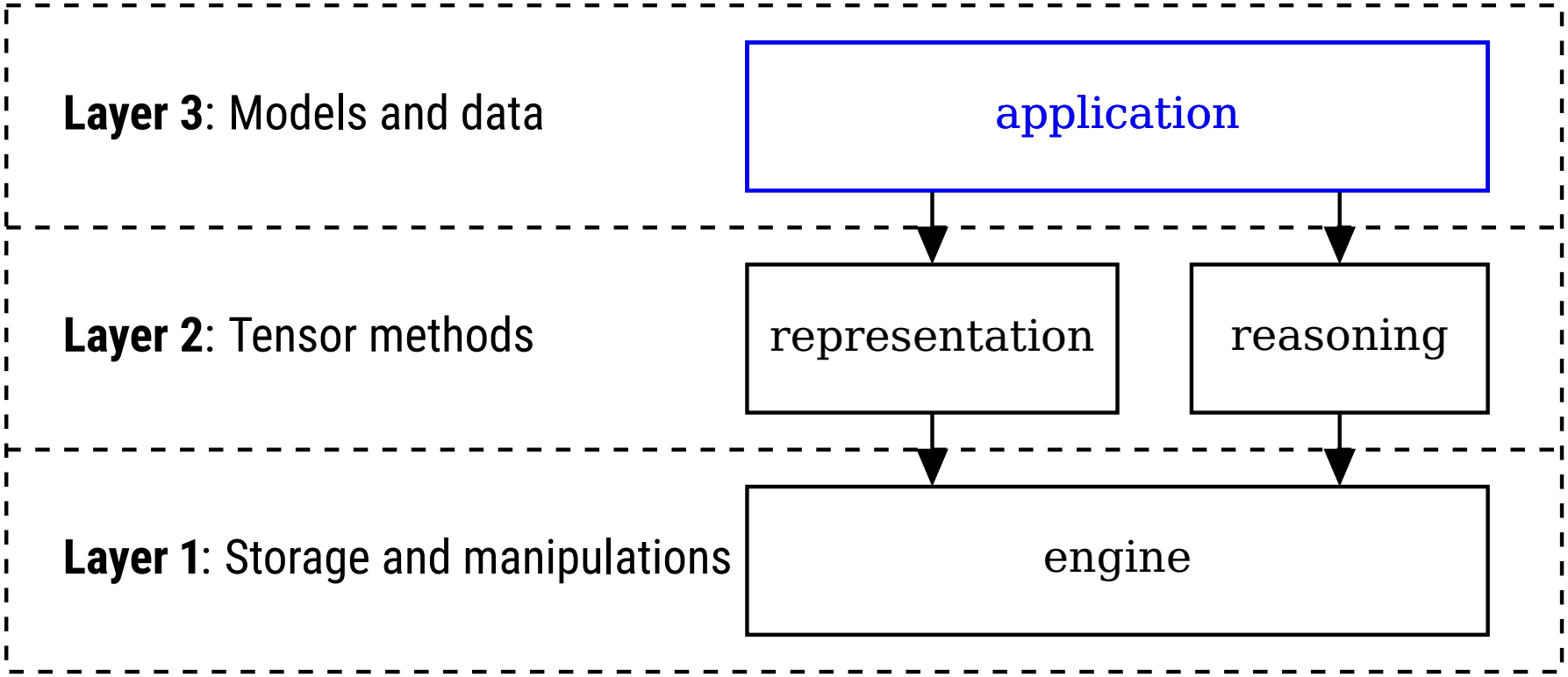
```
from tnreason import representation
basisEncodedCore =
↪  representation.create_basis_encoding_from_lambda(
   inShape, outShape, inColors, outColors, function)
```

- inShape, outShape, inColors, outColors: specifying the leg dimension and the names of the incoming variables ($X_{[d]}$) and the outgoing variables (Y_f)
- function: computes the state of the outgoing variables given the state of the incoming variables

Implementations in tnreason: Subpackage application

The subpackage application provides high-level models

- Interpretation of propositional syntax
- Training on empirical data (later)



Representation of Syntax

In tnreason, propositional syntax is represented by nested lists of strings, for example:

```
["and", ["not", "Rained"], ["imp", "Rained", "Wet"]]
```

Connectives $\circ$ get assigned a string $s(\circ)$:

Unary connective $\circ$	$s(\circ)$
$\neg$	*"not"*
$()$	*"id"*

Binary connective $\circ$	$s(\circ)$
$\wedge$	*"and"*
$\vee$	*"or"*
$\Rightarrow$	*"imp"*
$\oplus$	*"xor"*
$\Leftrightarrow$	*"eq"*

Any other string is interpreted by an atomic formula (a so-called propositional symbol).

Representation of Syntax

Having strings representing

- Connectives (by defined representations)
- Atomic Formulas (all other strings)

we represent formulas $f_1 \circ f_2$

$$s(f_1 \circ f_2) = [s(\circ), s(f_1), s(f_2)]$$

where we apply the conventions

- Connectives are at the 0th position in each list
- Further entries are either atoms as strings or encoded formulas itself

Examples of representations

Atomic variable Rained by

$s(\text{Rained})$ = `"Rained"`

Negative literal $\neg$Rained by

$s(\neg\text{Rained})$ = `["not","Rained"]`

Horn clause (Rained $\Rightarrow$ Wet) by

$s(\text{Rained} \Rightarrow \text{Wet})$ = `["imp","Rained","Wet"]`

Knowledge Base $\mathcal{KB} = (\neg\text{Rained}) \wedge (\text{Rained} \Rightarrow \text{Wet})$ by

$s(\mathcal{KB})$ =

```
["and", ["not", "Rained"], ["imp", "Rained", "Wet"]]
```

Basis encoding decompositions

The recursive structure of the nested lists `expression` $=s(f)$ is exploited in finding tensor network representations of f.

```
from tnreason.representation import formula_to_cores
↪  as ftc
coreDict =
↪  ftc.create_formula_computation_cores(expression)
```

The function creates a tensor networks of basis encodings to connectives by:

- If $s(f)$ a list, add $\beta^{s(f)[0]}$ and add tensors from recursive calls to the subformulas $s(f)[1:]$
- Return empty dictionary, when $s(f)$ is a string (then an atom)